Broken Tokens? Your Language Model can Secretly Handle No n - Ca noni cal Tokenizations

Brian Siyuan Zheng[♡] **Alisa Liu**[♡] **Orevaoghene Ahia**[♡]
Jonathan Hayase[♡] **Yejin Choi**[♠] **Noah A. Smith**[♡♣]
[♡]University of Washington [♣]Allen Institute for AI [♠]Stanford University
zhengbr@cs.washington.edu

Abstract

Modern tokenizers employ deterministic algorithms to map text into a single "canonical" token sequence, yet the same string can be encoded as many non-canonical tokenizations using the tokenizer vocabulary. In this work, we investigate the robustness of LMs to text encoded with non-canonical tokenizations entirely unseen during training. Surprisingly, when evaluated across 20 benchmarks, we find that instruction-tuned models retain up to 93.4% of their original performance when given a randomly sampled tokenization, and 90.8% with character-level tokenization. We see that overall stronger models tend to be more robust, and robustness diminishes as the tokenization departs farther from the canonical form. Motivated by these results, we then identify settings where non-canonical tokenization schemes can *improve* performance, finding that character-level segmentation improves string manipulation and code understanding tasks by up to +14%, and right-aligned digit grouping enhances large-number arithmetic by +33%. Finally, we investigate the source of this robustness, finding that it arises in the instruction-tuning phase. We show that while both base and post-trained models grasp the semantics of non-canonical tokenizations (perceiving them as containing misspellings), base models try to mimic the imagined mistakes and degenerate into nonsensical output, while post-trained models are committed to fluent responses. Overall, our findings suggest that models are less tied to their tokenizer than previously believed, and demonstrate the promise of intervening on tokenization at inference time to boost performance.[1]

1 Introduction

Tokenizers segment text into a sequence of discrete tokens in the language model's (LM) vocabulary. Most of today's LMs use deterministic subword tokenization, which produces a single canonical token sequence for a given piece of text, and further, for each whitespace-delimited word. One commonly discussed limitation of this approach is that, by mapping byte strings to symbolic token IDs, the orthographic makeup of tokens is obscured to the LM [52, 18]. This can be especially harmful for LM understanding of numbers [47, 64, 61] and morphologically rich languages [2, 25], and has motivated efforts to model text directly at the byte level [12, 70, 68, 63, 72, 46, 49, 1, 39].

To shed more light on this perceived limitation, in this work we study whether LMs can adapt *at inference time*, without any additional training, to a different tokenization scheme than the one they were trained with. While the tokenizer deterministically outputs a *canonical tokenization* of any text into tokens (usually by applying an ordered list of merge rules), *non-canonical tokenizations* of the same text using the same vocabulary are generally possible (see example in Figure 1). Here,

[1]Code is available at https://github.com/Brianzhengca/Tokenizer-Robustness.

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

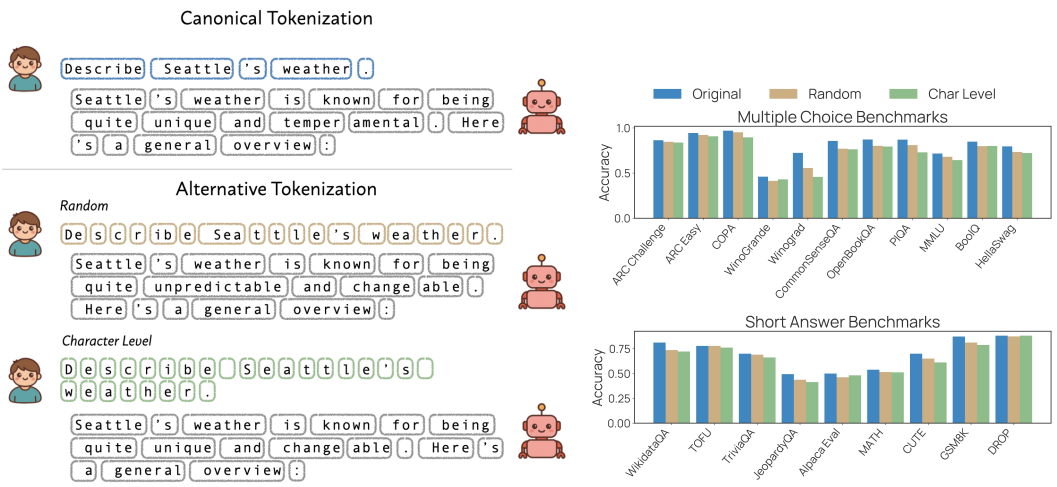

Figure 1: **Left:** An example of how LLAMA-3.1-8B-INSTRUCT responds when given **canonically tokenized** input, versus a **random tokenization** and **character-level tokenization**. The responses are surprisingly similar, demonstrating their ability to handle non-canonical tokenizations. Moreover, LMs generally respond with correctly tokenized output regardless of the tokenization scheme used for the context. **Right:** Performance of QWEN-2.5-7B-INSTRUCT across various benchmarks and tokenization schemes. Much of the original performance is preserved when given non-canonical tokenizations.

we evaluate how LMs trained with deterministic tokenizers behave when given non-canonical tokenizations of text. Surprisingly, we find that *instruction-tuned LMs across many model families are extremely robust* to non-canonical tokenizations (§2). For example, when evaluated across 20 benchmarks, QWEN-2.5-7B-INSTRUCT retains 93.4% of its original performance when presented with a random non-canonical tokenization, and 90.8% when presented with character-level tokens (see Figure 1). Thus, far from not understanding the makeup of their tokens, LMs are able to compose token sequences in entirely new ways at inference time [33].

This leads to an intriguing possibility: if LMs can process non-canonical tokenizations, can we use different tokenization schemes at inference time to improve performance? For instance, prior work has found that better segmentation of large numbers can improve accuracy on arithmetic [61, 56]. Indeed, we identify several settings where non-canonical tokenization schemes improve performance for LLAMA-3.1-INSTRUCT (§3): character-level tokenization brings up to +14% improvement on string manipulation and code understanding tasks, perhaps by granting LMs more direct access to orthographic cues. Meanwhile, right-aligned digit groups, which provide a consistent grouping of digits by powers of a thousand, improves arithmetic on large numbers by +33%. These performance gains are achieved without any model finetuning, pointing to the promise of tokenization as a means of inference-time control.

Finally, we investigate the origins of model robustness to non-canonical tokenizations (§4). Across multiple model families, we find that *pretrained-only* LMs consistently fail to produce fluent continuations given non-canonically tokenized context. By studying models at different stages of post-training, we identify that robustness arises during the supervised instruction-tuning (SFT) phase (§4.1). We then ablate differences between pretraining and SFT procedures and find that the separation of the instruction and response as distinct turns of conversation is key (§4.2). From here, we provide evidence for a plausible explanation: while both base and post-trained models grasp the semantics of non-canonical tokenizations, they also perceive them as containing misspellings (§4.2). Base models attempt to mimic the imagined mistakes and degenerate into nonsense, whereas post-trained models are not bound by the style of the instruction and thus able to produce fluent responses.

Overall, despite being trained with deterministic tokenization, instruction-tuned LMs readily accommodate new tokenizations at inference time, suggesting that LMs are less constrained by their tokenizer than previously believed [45]. Moreover, in settings where different representations of text are beneficial, we can intervene on tokenization at inference time for performance gains. We hope our work sheds new light on the discussion of strengths and limitations of tokenization, and points to the possibility of dynamically finding the optimal representation of text after pretraining.

Table 1: **Evaluated across many benchmarks, models are surprisingly robust to non-canonical tokenizations of the context.** We show the absolute drop in performance when given a randomly sampled non-canonical tokenization (**Rand** Δ) and character-level tokenization (**Char** Δ), relative to the canonical (**Canon**) tokenization. We also summarize the model's ability to retain performance across benchmarks and tokenization strategies (bottom).

	Qwen-2.5-7B-Instruct			Llama-3.1-8B-Instruct			Olmo-2-7B-Instruct		
Benchmark	Canon	Rand Δ	Char Δ	Canon	Rand Δ	Char Δ	Canon	Rand Δ	Char Δ
Multiple choice (MC)									
ARC-C	86.4	–1.80	–2.60	76.2	–14.10	–22.40	77.0	–37.40	–44.60
ARC-E	94.4	–2.20	–3.60	91.3	–12.60	–21.50	85.4	–37.00	–49.00
COPA	97.0	–1.80	–7.40	97.2	–9.60	–14.80	93.8	–21.40	–33.60
Winogrande	46.0	–4.60	–3.00	59.6	+2.00	–5.00	58.6	–7.80	–7.00
Winograd	72.4	–16.80	–26.60	74.4	–9.40	–13.00	72.4	–8.60	–19.00
CSQA	85.6	–8.60	–9.20	77.6	–11.40	–20.00	75.4	–31.60	–40.00
OpenbookQA	87.2	–7.00	–7.80	82.0	–13.80	–20.20	76.2	–30.80	–40.80
PIQA	87.0	–6.00	–14.00	84.0	–12.60	–18.40	78.2	–17.40	–25.00
MMLU	71.7	–3.70	–7.30	68.2	–11.60	–24.00	59.5	–16.30	–29.10
BoolQ	84.8	–5.00	–4.80	86.2	–19.20	–17.20	71.0	–4.00	–9.20
HellaSwag	79.6	–6.20	–7.40	68.6	–14.20	–23.40	68.0	–26.80	–39.80
Short answer (SA)									
WikidataQA	81.2	–7.60	–9.00	78.6	–12.40	–18.00	73.2	–28.80	–32.20
TOFU	77.8	+0.00	–1.70	82.1	+1.70	+0.80	82.9	–12.80	–23.90
TriviaQA	70.0	–1.00	–3.80	76.6	–9.80	–13.60	70.0	–22.20	–34.80
JeopardyQA	49.4	–5.60	–8.00	43.6	–2.20	–10.20	42.6	–21.60	–24.20
AlpacaEval	50.0	–3.70	–1.80	50.0	–5.70	–7.50	50.0	–2.10	–11.30
MATH	53.9	–2.40	–2.70	32.0	–4.20	–9.70	22.7	–5.20	–9.20
CUTE	70.0	–4.90	–8.80	68.0	–11.10	–15.30	55.3	–10.20	–5.70
GSM8K	87.3	–6.10	–8.50	82.0	–11.70	–16.00	73.9	–23.10	–35.80
DROP	88.2	–0.80	+0.00	88.8	–0.60	–5.00	77.0	–5.60	–7.60
Overall									
Avg MC Retention (%)		$92.4_{\pm 5.97}$	$89.2_{\pm 9.33}$		$85.6_{\pm 6.86}$	$76.8_{\pm 7.99}$		$71.2_{\pm 14.7}$	$59.2_{\pm 17.8}$
Avg SA Retention (%)		$94.6_{\pm 3.97}$	$92.7_{\pm 5.35}$		$90.3_{\pm 6.78}$	$82.7_{\pm 9.65}$		$75.4_{\pm 15.1}$	$65.4_{\pm 17.4}$
Avg Overall Retention (%)		$93.4_{\pm 5.15}$	$90.8_{\pm 7.81}$		$87.7_{\pm 7.05}$	$79.4_{\pm 9.05}$		$73.1_{\pm 14.7}$	$62.0_{\pm 17.5}$

2 Language Models are Robust to Non-Canonical Tokenizations

In our main experiments, we evaluate the robustness of LMs to non-canonical tokenizations by comparing their performance on downstream tasks when given different tokenizations of the input.

2.1 Background

Most LMs today, and all the models we study, use the *Byte-Pair Encoding* (BPE) [58] algorithm for tokenization. The BPE tokenizer is learned by splitting a corpus of text into bytes, which form the initial vocabulary, then iteratively merging the most frequent pair of tokens into a new token that is added to the vocabulary. To encode a new text, it is split into bytes, and the learned merges are applied in the same order. As a result, a BPE tokenizer always produces the same token sequence for the same text. Further, because BPE tokens do not cross whitespace boundaries, the same whitespace-delimited word is always represented with the same token or token sequence.

A natural observation is that given a tokenizer vocabulary, there exist many token sequences that decode to the same text. For instance, ␣*cat* could be tokenized as [␣cat], [␣, cat], [␣, c, at], [␣, c, a, t], etc. In general, the number of non-canonical tokenizations grows exponentially with the length of the text. Many previous works have argued that the probability of a string should be calculated as the sum of probabilities of all possible tokenizations [7, 9, 20]. However, less attention has been paid to how non-canonical tokenizations affect LMs in generative settings.

2.2 Setup

We consider two non-canonical tokenization schemes. (1) *Random tokenization* produces a tokenization (uniformly at random) from the set of tokenizations more granular than the canonical one. This can be achieved by recursively splitting individual tokens into a valid pair of tokens, similarly to

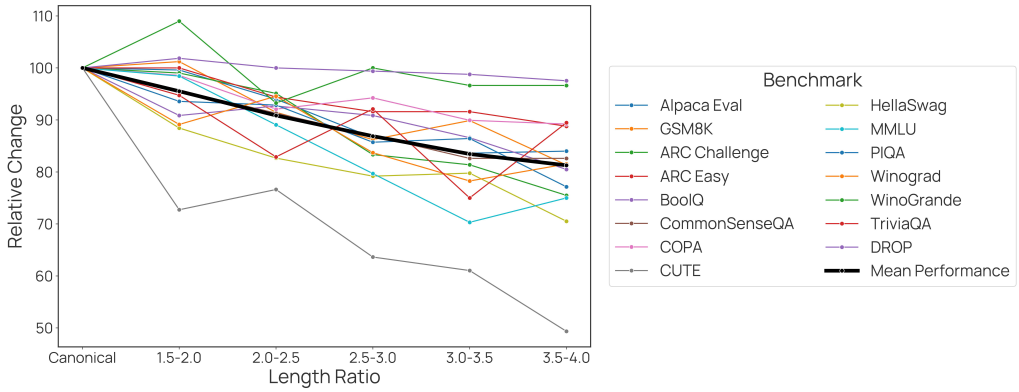

Figure 2: **Model performance generally declines as the tokenization becomes more granular.** We achieve variation in tokenization length using different values of p in BPE-dropout, and group tokenizations into buckets based on how many times longer it is than the canonical tokenization.

[60]; the pseudocode and a proof of correctness is provided in Appendix A. (2) *Character-level tokenization* decomposes the string into character tokens, i.e., using no subword token from the vocabulary. For text containing only English letters and punctuation (where each character is exactly one byte), this produces the most granular possible tokenization.

We consider three models, LLAMA-3.1-8B-INSTRUCT [43], OLMO2-7B-INSTRUCT [48], and QWEN-2.5-7B-INSTRUCT [53], which we evaluate on 20 benchmarks shown in Table 1. Please see §B.1 for further description of the datasets and evaluation setup.

2.3 Results

Shown in Table 1, while random tokenization consistently leads to worse performance compared to the canonical tokenization, the effect is small. On average across benchmarks, QWEN-2.5 retains 93.4% of its performance when given random tokenization, followed by LLAMA-3.1 at 87.7% and OLMO-2 at 73.1%. The performance drops further with character-level tokenization, with retention of 90.8%, 79.4%, and 62.0% for the three models, respectively. This ranking of models in terms of retention is consistent with their ranking in absolute accuracy (under canonical tokenization), suggesting that stronger models are generally more robust to non-canonical tokenization strategies.

We also observe that all models retain performance better on short answer (SA) benchmarks (where the model generates an output in free-form) compared to multiple choice (MC) benchmarks (where the model is instructed to directly output the correct answer choice). In addition, LMs consistently *produce* correct token sequences even when conditioning on non-canonical tokenizations. We hypothesize that, in the SA setting, models benefit from eventually conditioning on recent correctly-tokenized context.

2.4 Analysis: How does granularity of the tokenization affect robustness?

We next study whether tokenization fine-grainedness correlates in general with model robustness. We measure the fine-grainedness of a given non-canonical tokenization by how many times longer it is (in tokens) than the canonical tokenization, which we call the "length ratio." Finer-grained tokenizations have higher ratios, while coarser ones have ratios closer to 1. We produce tokenizations with diverse length ratios by applying BPE dropout [52] with $p \in [0.1, 0.2, ..., 0.9]$, which controls the probability with which each merge is dropped. (High p leads to finer-grained segmentations, and $p = 0.0$ corresponds to conventional BPE.)

Figure 2 shows the relationship between the length ratio and the average performance retention relative to canonical tokenization, with finer-grained tokenization generally leading to worse performance. When performance retention is averaged across tasks, the negative correlation is statistically significant under Kendall's τ with $p = 0.003$.

Table 2: **Examples from tasks** we construct where non-canonical tokenizations lead to improved performance for LLAMA-3.1-7B-INSTRUCT (§3).

Counting characters: `Count the number of the letter 'r' in the word strawberry.`
Acronyms: `Come up with a sequence of words where the first letters would form this acronym: isman`

Codeline Description: `What does the following code do:`
`{code block}`
`A. Counts paths from a point to reach Origin`
`B. Program to check if a matrix is symmetric`
`C. Longest subsequence from an array of pairs having first element increasing and second element decreasing .`
`D. Count the number of strings in an array whose distinct characters are less than equal to M`

Arithmetic: `8492079913 + 4877278482 =`

3 Can non-canonical tokenizations *improve* model performance?

If LMs can process non-canonical tokenizations, this points to the exciting possibility that tokenization schemes can be modified completely at inference-time. This would be useful if, in certain settings, there exists a better representation of text than what the tokenizer produces. In this section, we develop a suite of tasks that intuitively require understanding of the orthography of the text, and show that LLAMA-3.1-8B-INSTRUCT performs better under non-canonical tokenization schemes.

3.1 Tasks

Please see Table 2 for an example question in each task and Table B.2 for further details on dataset construction. For all tasks except Arithmetic, we use character-level tokenization.

Counting Characters This task asks the model to count the number of occurrences of the most common letter in 5-10 character tokens in LLAMA-3.1's vocabulary, and contains 1001 samples.

Acronyms This task asks models to generate a list of words whose first letters form a given acronym. We construct 3594 5-letter acronyms by sampling each letter uniformly at random from the alphabet.

Code Description For a more real-world application, we construct a task where the model is given a code snippet and asked to identify the function of the code in natural language from four MC options. The setup is inspired by the Codeline Description task from BIG-Bench [5], but to increase the difficulty we use more complex code snippets and corresponding natural language descriptions from XLCoST [74]. To collect incorrect answers, we sample three other code descriptions from the dataset. This task contains 4800 samples across 6 programming languages.

Arithmetic Prior work has suggested that arithmetic is difficult for LMs in part due to poor segmentation of digits [47, 64]. We curate a simple arithmetic dataset by constructing addition and subtraction tasks for 10 digit numbers. Here, we use a different segmentation strategy. The LLAMA-3.1 tokenizer segments numbers into groups of three left-to-right (e.g., *1000000* is encoded as ["100", "000", "0"]), due to the pretokenization regular expression looking for matches greedily from the left. Inspired by [61], we instead segment digits into groups of three right-to-left (e.g., ["1", "000", "000"]). This task contains 1000 addition and subtraction questions in total.

3.2 Results

Shown in Table 3, in all the tasks we construct, the non-canonical tokenization strategy leads to substantially better performance compared to the canonical tokenization. In particular, we observe a +14.3% improvement on code description and +33.7% on arithmetic. Our results show that the tokenization scheme used in training is not necessarily the optimal one at inference-time, and

Table 3: **On several tasks, LLAMA-3.1-8B-INSTRUCT achieves *better* performance when using a non-canonical tokenization scheme**. For the first four tasks, the input is tokenized at the character level; for Arithmetic, we segment digits into groups of three digits from right to left (instead of the usual left to right). On all tasks, we observe a large performance improvement from using the alternative tokenization scheme.

Task	Canonical	Alternative	Δ
Counting Characters	66.5	**73.5**	+6.99
Acronyms	49.7	**57.4**	+7.74
Code Description	68.6	**82.9**	+14.3
Arithmetic	36.5	**70.2**	+33.7

replacing them with intuitively meaningful tokenizations can bring substantial performance gains. We leave automatically identifying the optimal tokenization as a promising direction for future work.

4 Investigating the Source of Robustness

Thus far, our experiments have used post-trained "instruct" models. In this section, we find that pretrained-only models are actually unable to produce fluent continuations of unusually tokenized context (§4.1), perform ablations to identify the conditions enabling robustness (§4.2), and finally provide support for an explanation of why generative robustness arises during post-training (§4.3).

4.1 When does robustness appear in model training?

We first quantify the robustness of models at different stages of the model development pipeline by using the OLMO2 and TULU3 [36] model families which include the base, SFT, DPO, and final instruct models. For simplicity, we focus on AlpacaEval and use character-level tokenization. For base models, we construct the prompt by placing the instruction in a question-answer template (`Question: {instruction}\nAnswer:`). We define three simple measures of generation quality.

Spelling We measure the proportion of (whitespace-delimited) words in the generation that can be found in a collection of the top 10,000 most common English words.[2]

Grammaticality We use LanguageTool's grammar checker[3] to count the number of grammatical mistakes, which we normalize by the number of words in the generation and subtract from 1 to produce a grammaticality score where higher is better.

Win rate To measure overall generation quality, we use `alpaca_eval_gpt4` as an LM judge in the AlpacaEval framework and report the win rate of the generation given alternative against canonical tokenizations of the context. Unlike the previous two metrics, this measures not only the quality of the generation but also its relevance to the context.

Shown in Figure 3, the base models of OLMO2 and LLAMA-3.1 are both unable to produce sensible output conditioned on character-level tokenizations of context, scoring at best 0.317 on spelling and 0.260 on grammaticality. Qualitatively, generations are extremely difficult to parse and often involve odd character substitutions and repetitions (e.g., `Yoou`, `haviin`). Despite this, they sometimes reflect an understanding of the prompt. Consider, for example,

```
Question: I like to host guests at my home from time to time [...] Can
you give me a recipe for Canjeero?
Answer: I aam glade tio hear tio hear tio hear tio hear that yoou enjoy
haviin gauests at yoour hoome an tio keeep tio keeep tio keeep
```

[2] https://github.com/first20hours/google-10000-english

[3] https://github.com/languagetool-org/languagetool

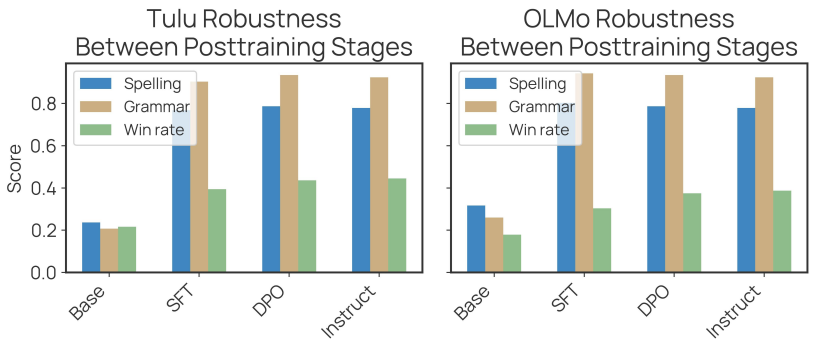

Figure 3: **Pretrained-only models completely fail to generate coherent output conditioned on non-canonical tokenizations of context; robustness is gained in the SFT stage.** We evaluate the spelling, grammaticality, and AlpacaEval win rate of model generations. Note that since TULU3 uses LLAMA-3 as the base model, its base scores are computed using LLAMA-3's base model scores.

In contrast, the post-trained models are more robust across all three metrics, with *much of the improvement coming from the SFT stage alone*.

4.2 Why do instruction-tuned models become robust?

We first replicate the finding from §4.1 that SFT yields robustness to non-canonical tokenizations by finetuning the LLAMA-3.2-1B base model on the TULU 3 SFT PERSONAS INSTRUCTION FOLLOWING dataset. Then, we perform the following interventions on the SFT training data and procedure to shed light on the possible source.

Gradient over full sequence SFT on instruction-response pairs conventionally uses a loss mask over the instruction tokens, so that only the response tokens contribute to the loss. We remove this loss mask and instead compute gradients over the entire instruction and response.

Question/answer template We replace the chat template with a simple question-answer template, `Question: {instruction} Answer: {response}`, both for training and evaluation.

Removing the chat template We remove the chat template by concatenating the instruction and response without any special formatting. In evaluation, we again provide the instruction alone.

Removing the instruction After SFT training, the LM's goal is no longer to continue a given text prefix, but rather to generate a response to the given instruction. To ablate the nature of the data itself, we take only the responses from the SFT data, and randomly split each into a new "prompt" and "response," which we format with the SFT template.[4] At test time, we similarly provide an incomplete response within "instruction" tags. Since the purpose of the passage is generally inferrable from the first few words of the gold response ("*Sure, here's a recipe for Kubdari...*"), we are able to evaluate generated responses under the same AlpacaEval framework.

Our results are shown in Figure 4. We replicate the finding that SFT (**No ablation**) leads the model to be able to handle non-canonical tokenizations. This persists when computing gradients over the entire instruction and response (**Full gradient**) so that the training procedure matches regular pretraining. Replacing the original chat template with a simple question-answer template (**QA template**) also maintains model robustness. However, the usage of a template is crucial — when directly concatenating the instruction and response (**Removing chat template**), the model fails to produce coherent generations, with the spelling score dropping from 0.786 in the no ablation setting to 0.0698. Inserting the chat template into pretraining-style data (**Removing the instruction**) also does not yield robustness, with a spelling and grammaticality scores remaining low at 0.181 and 0.158,

[4]We match the instruction length distribution by counting the number of tokens n in the original instruction, and formatting the first n tokens of the response as the new "instruction."

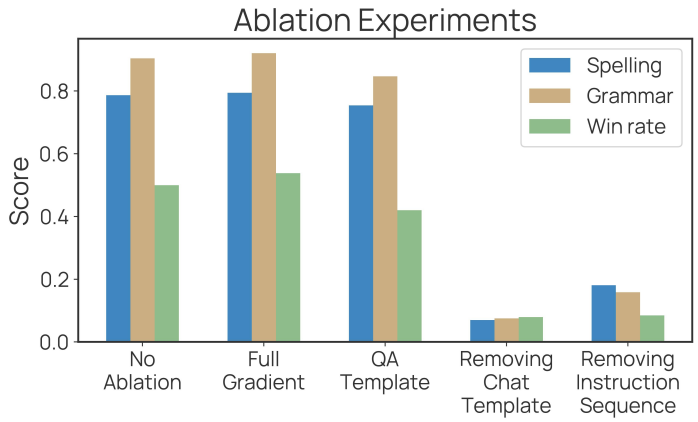

Figure 4: Ablations on the SFT training data and procedure indicate that the separation of the context and expected continuation — as different turns of dialogue demarcated with a special token — is key to robustness to non-canonical tokenizations.

Table 4: Both base and instruct models from the LLAMA-3.1-8B family recognize words represented with non-canonical tokenizations (performing well on **Word Repeat**), but incorrectly perceive that there are misspellings (performing at random on **Identifying Misspellings**).

	Word Repeat	Identifying Misspellings
Base model	90.8	48.2
Instruct model	92.0	55.8

respectively. Overall, these findings suggest that in order for the LM to generate fluent continuations given non-canonical tokenizations, the context and expected continuation need to represent separate turns of dialogue, and additionally, be demarcated with a special template.

4.3 Disentangling understanding from generation

One plausible explanation for our findings thus far is that both base and instruction-tuned models grasp the semantics of non-canonical tokenizations, yet falsely perceive them as containing misspellings. While base LMs attempt to faithfully continue these mistakes and degenerate into nonsensical output, instruction-tuned models are trained to provide fluent responses regardless of the instruction, leading to the results observed in §4.1. To test this hypothesis, we construct two simple tests:

1. **Word Repeat**: To determine if a model perceives the meaning of a word with non-canonical tokenization, we prompt the model to repeat a given word (while correcting any typos).
2. **Identifying Misspellings**: To determine if a model perceives a misspelling, we ask it to identify the word with a misspelling among two options: a (correctly tokenized) misspelling of a word and an non-canonical tokenization of that word (correctly spelled).

Results are shown in Table 4. Consistent with our hypothesis, we find that both the base and instruct models from the LLAMA-3.1-8B family score highly ($> 90\%$) on Word Repeat. This means that the base model, despite its poor performance in §4.1, actually recognizes the correct form of non-canonical tokenizations as well as its post-trained counterpart. In addition, both models perform at random when asked to distinguish non-canonical tokenization from true misspellings. In other words, the instruct model produces fluent responses (§2) while interpreting the instruction as heavily misspelled! While instruct models evidently overcome this, the base model likely attempts to mimic the (perceived) idiosyncratic surface form, thus producing nonsensical (yet sometimes relevant) outputs.

5 Related Work

The extent to which LMs are limited by their tokenization is a topic of much debate, with the story evolving as LMs become larger and more capable.

Character-level understanding in tokenizer-based models It is commonly argued that tokenization obscures orthographic information about tokens from the LM, leading to unexpected failures [18, 8, 67]. As a result, there have been many efforts towards linguistically-informed tokenization that make derivational, compound, and morphological boundaries within words explicit [35, 25, 26, 71, 4]. Similarly, BPE-dropout [52] and related methods [60] introduce variation in training in how a given string is tokenized in order to make models more robust to rare, misspelled, or unseen words.

However, there is other evidence that LMs naturally overcome these limitations. For instance, token embeddings have been found to robustly encode character-level information, especially in larger models [34, 27]. This may be because word variants that do not share tokens in common (consider e.g., `[␣dictionary]` and `[␣diction, aries]`, as tokenized by GPT-2) incentivize the model to learn spelling as a general solution to understanding their relations [34]. Other works argue that LMs maintain an implicit vocabulary, and can compose arbitrary token sequences (including non-canonical ones) into useful higher-level representations [19, 33]. Even in domains like biomedical text where terms are highly agglutinative, using tokenizers that segment on meaningful components does not lead to improved models [30]. Recent works have even found that coarser *superword* tokenization [40, 57], which capture common word sequences in a single token, preserve character-level understanding while providing benefits in compression and downstream performance.

Our work informs this conversation by showing that LMs can effectively leverage character-level knowledge of their tokens and glean potential benefits of improved representation at inference time.

Partial token problem A related but distinct problem is the *partial token problem* (also known as *tokenization bias* or the *prompt boundary problem*) where the prompt ends with the prefix of a valid token, causing the model to assign unexpectedly low probability to the completion of that token. Many works have found that this continues to compromise a serious failure mode for frontier LMs [69, 51, 41, 66, 22]. In particular, DEEPSEEK V3 [15] aims to improve robustness to partial punctuation tokens by randomly splitting some proportion of multi-punctuation tokens into smaller tokens during training, though they do not present experiments with this ablation. We note that these results are not inconsistent with ours — together, they suggest that while models are very unlikely to *generate* non-canonical tokenizations, they can nonetheless understand them in the context history.

Non-canonical tokenizations It has long been recognized that there are many possible ways to segment a string into tokens with a fixed vocabulary [10], which in principle should be considered in the calculation of a string's likelihood [7, 9, 20]. Previous work has briefly touched on non-canonical tokenization in the context of self-supervised evaluation [28] and defense against adversarial attacks [29]. In contemporaneous work, Geh et al. [21] also show that non-canonical tokenizations can be constructed adversarially to trigger unsafe completions. In contrast, we provide a more systematic study of LM robustness using benchmark evaluations and additionally study its source.

Somewhat relatedly, other works have provided algorithms for sampling at the character- or byte-level from tokenizer-based LMs [50, 65, 3, 22]. Together, these directions suggest that despite being trained with one deterministic tokenization scheme, LMs can both condition on and produce token sequences over a different (sub)vocabulary.

6 Conclusion

Despite being trained with deterministic tokenization algorithms, we show that instruction-tuned language models are surprisingly robust to token sequences not seen in training. In certain domains, such as arithmetic or code, more intuitively meaningful tokenizations can even be swapped-in at inference time for improved performance. We analyze the source of this robustness, and find that while the base and instruct models both perceive the semantics of non-canonical tokenizations, only instruct models are capable of providing fluent continuations. Our work demonstrates a way in which LMs are not necessarily tied to tokenizer they were trained with, and highlights the potential of finding more optimal representations of text after pretraining.

Acknowledgments

We would like to thank Joseph An and Ricky Koppolu, as well as the broader UW NLP community, for helpful conversations about this work. AL and JH are supported by the NSF Graduate Research Fellowship.

References

[1] O. Ahia, S. Kumar, H. Gonen, V. Hofmann, T. Limisiewicz, Y. Tsvetkov, and N. A. Smith. MAGNET: Improving the multilingual fairness of language models with adaptive gradient-based tokenization. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=1e3MOwHSIX.

[2] C. Arnett and B. Bergen. Why do language models perform worse for morphologically complex languages? In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 6607–6623, 2025.

[3] B. Athiwaratkun, S. Wang, M. Shang, Y. Tian, Z. Wang, S. K. Gonugondla, S. K. Gouda, R. Kwiatkowski, R. Nallapati, P. Bhatia, and B. Xiang. Token alignment via character matching for subword completion. In L.-W. Ku, A. Martins, and V. Srikumar, editors, *Findings of the Association for Computational Linguistics: ACL 2024*, pages 15725–15738, Bangkok, Thailand, Aug. 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.929. URL https://aclanthology.org/2024.findings-acl.929.

[4] T. Bauwens and P. Delobelle. BPE-knockout: Pruning pre-existing BPE tokenisers with backwards-compatible morphological semi-supervision. In K. Duh, H. Gomez, and S. Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5810–5832, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.324. URL https://aclanthology.org/2024.naacl-long.324.

[5] B. bench authors. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023. ISSN 2835-8856. URL https://openreview.net/forum?id=uyTL5Bvosj.

[6] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi. Piqa: Reasoning about physical commonsense in natural language. In *Thirty-Fourth AAAI Conference on Artificial Intelligence*, 2020.

[7] K. Cao and L. Rimell. You should evaluate your language model on marginal likelihood over tokenisations. In M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 2104–2114, Online and Punta Cana, Dominican Republic, Nov. 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.161. URL https://aclanthology.org/2021.emnlp-main.161.

[8] Y. Chai, Y. Fang, Q. Peng, and X. Li. Tokenization falling short: On subword robustness in large language models. In Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1582–1599, Miami, Florida, USA, Nov. 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.86. URL https://aclanthology.org/2024.findings-emnlp.86.

[9] N. Chirkova, G. Kruszewski, J. Rozen, and M. Dymetman. Should you marginalize over possible tokenizations? In A. Rogers, J. Boyd-Graber, and N. Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1–12, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-short.1. URL https://aclanthology.org/2023.acl-short.1.

[10] K. W. Church. Emerging trends: Subwords, seriously? *Natural Language Engineering*, 26(3): 375–382, 2020. doi: 10.1017/S1351324920000145.

A Random Non-Canonical Tokenization

In this section, we provide our algorithm for producing a random non-canonical tokenization, and a proof that each non-canonical tokenization that is *more fine-grained* than the canonical one has equal probability of being output.

A.1 Algorithm

We will split each token in a canonical tokenization into smaller tokens (that each exists in the tokenizer's vocabulary). We formulate our problem as: Given a valid token t, and a set of vocabulary $\mathcal{V}$, construct a sequence of tokens seq using tokens that exist in $\mathcal{V}$ and form t when concatenated together. We produce seq using a recursive algorithm. Since there can be many possible seq for each t, we need to randomly choose one and guarantee that each possible seq is chosen with equal probability. We achieve this by considering recursion as producing a tree. Each path down the tree corresponds to one possible way to segment t. Each node of the tree represents a segmentation state where we have chosen some number of sub-tokens. At each node, we weigh the choice of which child node to visit by the number of leaves in the sub-tree that is rooted at each child node. This guarantees that each path down the tree is chosen with equal probability since the number of paths down a tree is equal to the number of leaf nodes in that tree. The pseudocode for the algorithm is in 1.

A.2 Proof

Goal: To prove that the random segmentation algorithm chooses one valid segmentation from all possible valid segmentations with uniform probability.

Notation: Let $W(i)$ denote the number of valid segmentation completions (i.e., the number of leaves in the recursive tree) for the substring starting at index i. In particular, $W(|token|) = 1$. Note that $W(i)$ is calculated by the memoized recursive function $countSegments(i)$, which calculates the number of leaves of the subtree rooted at i.

Base Case: Consider the node corresponding to $i = |token|$ (the end of the token). Here, there is exactly one valid segmentation (the empty segmentation), so the algorithm returns it with probability 1. That is, every segmentation (in this case, the only one) is chosen with probability

$$\frac{1}{W(|token|)} = \frac{1}{1} = 1.$$

Thus, the base case holds.

Inductive Hypothesis: Assume that for any node corresponding to an index j with $j > i$ (i.e., deeper in the recursion tree), every complete segmentation (leaf) in the subtree rooted at j is chosen with probability

$$\frac{1}{W(j)}.$$

Inductive Step: Now consider a node corresponding to index i (with $i < |token|$). Suppose that from i there are k valid branches corresponding to choosing substrings that end at indices $j_1, j_2, \ldots, j_k$, where for each j we have $i < j \leq |token|$ and the substring $token[i : j]$ is in the vocabulary. By definition,

$$W(i) = \sum_{r=1}^{k} W(j_r).$$

The algorithm selects the branch from i to a specific child j with probability

$$P(i \to j) = \frac{W(j)}{W(i)}.$$

Once branch $i \to j$ is chosen, by the inductive hypothesis every complete segmentation (leaf) in the subtree rooted at j is chosen with probability

$$\frac{1}{W(j)}.$$

Thus, the probability $P(S)$ of obtaining a particular complete segmentation $\mathcal{V}$ that starts at i by first taking the branch $i \to j$ and then following a specific path in the subtree rooted at j is

$$P(S) = \frac{W(j)}{W(i)} \cdot \frac{1}{W(j)} = \frac{1}{W(i)}.$$

Since the factor $W(j)$ cancels, the probability $P(S)$ is independent of the particular child j chosen.

Conclusion: By the inductive step, every complete segmentation (leaf) in the subtree rooted at any index i is chosen with probability $\frac{1}{W(i)}$. In particular, when $i = 0$ (the start of the token), every valid segmentation of the entire token is selected with uniform probability $\frac{1}{W(0)}$. This completes the proof.

Algorithm 1 Random Token Segmentation

function COUNTSEGMENTS($start$) ▷ Cached (using memoization)
  if $start = |token|$ **then**
    return 1 ▷ Reached end; valid segmentation
  end if
  $total \leftarrow 0$
  for $end \leftarrow start + 1$ **to** $|token|$ **do**
    $substring \leftarrow token[start : end]$
    if $substring \in vocabulary$ **then**
      $total \leftarrow total +$ COUNTSEGMENTS(end)
    end if
  end for
  return $total$
end function
function BUILDSEGMENTS($start$)
  if $start = |token|$ **then**
    return $\varnothing$ ▷ Empty segmentation
  end if
  $validSegments \leftarrow []$
  $weights \leftarrow []$
  for $end \leftarrow start + 1$ **to** $|token|$ **do**
    $substring \leftarrow token[start : end]$
    if $substring \in vocabulary$ **then**
      $segCount \leftarrow$ COUNTSEGMENTS(end)
      if $segCount > 0$ **then**
        Append $substring$ to $validSegments$
        Append $segCount$ to $weights$
      end if
    end if
  end for
  if $validSegments$ is empty **then**
    return $\varnothing$
  end if
  $chosenSegment \leftarrow$ weightedRandomChoice($validSegments, weights$)
  return $[chosenSegment] \parallel$ BUILDSEGMENTS($start + |chosenSegment|$) ▷ Concatenate chosen segment with segmentation of remaining token
end function
procedure SEGMENTTOKEN($token, vocabulary$)
  if COUNTSEGMENTS(0) $= 0$ **then**
    return $\varnothing$ ▷ No valid segmentation exists
  else
    return BUILDSEGMENTS(0)
  end if
end procedure

B Evaluation Details

B.1 General benchmarks

For short-answer benchmarks, the system prompt is:

```
You are a helpful assistant.
```

For multiple-choice benchmarks, the system prompt is:

```
You are a helpful assistant. For the following multiple choice questions,
return the answer only, without any additional reasoning or explanation.
```

MATH MATH is a dataset composed of fairly difficult, competition level math problems [24]. The test set is composed of short answer problem that describe some scenario and asks the model to output a mathematically correct answer.

GSM8K GSM8K is a dataset consisting of relatively simple math questions that would appear in grade school math exams [14]. For GSM8K, the evaluations were done in the same manner as MATH.

MMLU MMLU is a benchmarks comprising of multiple choice questions from a wide variety of subjects. [23] We sampled 500 questions from MMLU for our evaluation. We instructed the model to only output one answer to each question without any explanation.

Alpaca Eval Alpaca Eval is an evaluation benchmark where generations from language models against given prompts are compared and judged by an annotator model. [17] The metric used was raw winrate of the perturbed model as judged by a language model. The annotator we used was *alpaca_eval_gpt4*, which has been shown to have the highest Spearman and Pearson correlation coefficient with human annotators.

ARC Challenge and ARC Easy Contains multiple choice questions with four options each, taken from grade school science exams [13]. ARC Easy is tests basic science knowledge while ARC Challenge requires some procedural reasoning.

BoolQ Contains true or false questions along with a context passage that provides the answer to the question. [11]

CommonsenseQA Contains multiple choice questions with five options each that requires common sense knowledge to answer. [62]

COPA Contains multiple choice questions with two options each that tests knowledge of cause and effect. [54]

CUTE Contains questions that require the model to manipulate sentence-level, word-level, and character-level structure for strings. [18]

DROP contains questions that potentially require reasoning multiple pieces of information present in a given passage. [16]

HellaSwag contains multiples choice questions with four options each that asks for the most natural continuation to some given context. [73]

JeopardyQA contains short answer questions from the "Jeopardy!" game show. [32]

OpenbookQA contains multiple choice questions with four options each that require some multi-step and common sense reasoning. [44]

Table 5: System prompt for tasks in §3. See Table 2 for example instructions.

Counting characters: `You are a helpful assistant. The following prompt will ask you to return a sequence of words. Only return the sequence, separated by spaces. Do not provide any additional text or explanation.`
Code Description: `You are a programming assistant trained to analyze and interpret code snippets. When provided with a code snippet and a set of answer choices (A, B, C, or D), your task is to evaluate the code, determine its behavior, and select the answer that best describes this behavior. Your response must be a single letter: A, B, C, or D. Do not provide explanations or additional text unless explicitly requested.`
Arithmetic: `You are a computational assistant trained to evaluate arithmetic operations. When provided with an arithmetic expression, calculate the result and round it to the nearest integer. Respond only with the rounded result, without any additional text or explanation.`

PIQA contains multiple choice questions that require reasoning about the physical world. [6]

TriviaQA contains short answer questions that requires knowledge of the world. [31]

Winograd contains multiple choice questions with two options that asks to determine what a pronoun might refer to. Answering these questions require knowledge of commen sense and surrounding context. [37]

Winogrande contains questions in the same format of Winograd but there are more questions and the questions are harder. [55]

TOFU contains general short answer questions that tests the model's ability to process world knowledge. This is the retain set of the task of fictitious unlearning dataset. [42]

WikidataQA require models to complete factual statements. [5]

B.2 Constructed Benchmarks

In this section, we provide more detail on how datasets we use in §3 are constructed.

Count Characters Task The prompt asks the model to count the number of occurrences of a given character in a 10-character word; we always use the most frequently occurring character. Evaluation was done, similar to GSM and MATH, by finding the last number in the generated response. Generations without any numbers are considered incorrect.

Generate Acronym Task The model is asked to generate a sequence of words whose first letters form a randomly sampled five character string. For evaluation, we take the first character of each whitespace-delimited word and check if it matches the desired acronym.

Codeline Description Task The model is asked to comprehend a piece of code and choose the best description from four options.

Arithmetic Task The model is asked to perform addition or subtraction with 10 digit numbers. We use regex to extract numbers from the generation, which are then compared to the ground truth answer.

B.3 Metrics of generation quality

Here we provide additional details on the metrics defined in §4.1.

Table 6: Data format of ablations in §4.2.

No ablation: <|user|>Provide a detailed analysis of Candace Parker's defensive techniques in her recent games, excluding the words "aggressive" and "blocking", in the format of a sports commentary script. <|assistant|>[Sports Commentary Script] [Opening Scene...
QA Template: Question: Provide a detailed analysis of Candace Parker's defensive techniques in her recent games, excluding the words "aggressive" and "blocking", in the format of a sports commentary script. Answer: [Sports Commentary Script] [Opening Scene...
Removing the chat template: Provide a detailed analysis of Candace Parker's defensive techniques in her recent games, excluding the words "aggressive" and "blocking", in the format of a sports commentary script. [Sports Commentary Script] [Opening Scene...
Removing the instruction: <|user|>[Sports Commentary Script] [Opening Scene: A packed basketball arena, with fans eagerly awaiting the analysis of Candace Parker's recent performances on the court.] Commentator 1: Welcome back, basketball fans! <|assistant|>Tonight, we're diving into the defensive prowess of Candace Parker...

Spelling We use the top 10000 most frequently appearing English words in Google's trillion word corpus. We only consider words with more than one character. This is because sometimes base models will repeatedly generate the same letter, and since all English letters are in the word list, the generation would receive a high score.

Grammaticality One drawback with this evaluation method is that oftentimes the model would repeat the same letter over and over again, or start counting numbers. In both of these cases, there are no detected grammar mistakes, however they are still obviously gibberish. Therefore, we only calculate grammaticality scores for generations that receive a score ≥ 0.5 on spelling; otherwise, we give it a grammaticality score of 0.

Win rate Similar to evaluation in §2, we also used `alpaca_eval_gpt4` as the evaluator and report raw win rate. In 4.1, the win rate is calculated against generations conditioned on input with canonical tokenization. In 4.2, the win rate is against generations from the **No Ablation** setting when also given character-level tokenization. By construction, the win rate of the **No Ablation** setting itself is 50%.

B.4 Ablation Settings

For ablations on the data format, see examples of formatted data in Table 6. Our finetuning code was forked from `allenai/open-instruct`. The exact finetune recipe is given below:

- Setup: 8 L40S GPUs
- Gradient Accumulation Steps: 20
- Per Device Train Batch Size: 2
- Mixed Predision: bf16
- Max Seq Length: 4096
- Learning Rate: 5e-06
- LR Scheduler Type: Linear
- Warmup Ratio: 0.03
- Weight Decay: 0
- Epochs: 1
- Seed: 123

Table 7: QWEN-2.5-7B-INSTRUCT is robust to character-level tokenization of Chinese text.

Benchmark	Canon	Char
Chinese MMLU	77.8	74.2
Chinese GSM	78.8	76.8

B.5 Disentangling understanding from generation

For these tasks, we use 500 words randomly sampled from Google's 10000 English word list[5].

Word Repeat An example prompt is shown below.

```
Repeat each word directly, while correcting any typos.

Question: guarantees
Answer: guarantees

Question: revelation (character-level tokenization)
Answer:
```

Identifying Misspellings We obtain the misspelled word by randomly adding, removing, or substituting a single character from the word. An example prompt is shown below.

```
Question: Which of the two words contains a misspelling? Respond directly
with the answer option.

Question:

A. guarantees
B. garantees

Answer: B

{9 more in context examples}

Question:

A. farmer (character-level tokenization)
B. farme (canonical tokenization)
```

C Additional Results

C.1 Evaluation on Chinese Benchmarks

We also investigate how robust language models are given character-level tokenization of text in Chinese as evaluated on two tasks, Chinese GSM [59] (part of multilingual GSM benchmarks) and Chinese MMLU [38]. Note that since each Chinese character is usually represented with three bytes under UTF-8 encoding, this is not equivalent to byte-level tokenization. We focus on QWEN-2.5-7B-INSTRUCT as LLAMA-3.1-8B-INSTRUCT and OLMO-2-7B-INSTRUCT do not officially support Chinese. As shown in Table 7, we observe a similar robustness on Chinese, with performance dropping by only $\sim 3\%$ on each task.

[5] https://github.com/first20hours/google-10000-english/blob/master/google-10000-english.txt